

Pázmány Péter Katolikus Egyetem
Információs Technológiai és Bionikai Kar

Önálló laboratórium

2025/2026/2 félév

**Multithreadelt C++ alapú lokális cache szerver
teljesítményvizsgálata és optimalizációs lehetőségeinek
elemzése**

Hallgató:

Juhász Bence

Szak: IANI-MI – Mérnökinformatikus

BSc

Témavezető:

Dr. Feldhoffer Gergely

Budapest, 2026

Tartalomjegyzék

1. Feladat ismertetése	3
2. Bevezetés	4
2.1. A téma motivációja	4
2.2. Elméleti háttér	4
2.3. A vizsgált hipotézisek	4
3. A konkrét munka ismertetése	5
3.1. Mérési környezet és módszertan	5
3.2. A szerver implementációja	5
3.3. H1: Telítési pont a kliensszám függvényében	5
3.3.1. Hipotézis és felállítása	5
3.3.2. Eredmények	5
3.3.3. Diskusszió	6
3.4. H2: Worker pool méretének optimuma	6
3.4.1. Hipotézis és felállítása	6
3.4.2. Eredmények	6
3.4.3. Kontrollmérés 2 magos pinninggel	6
3.4.4. Diskusszió	6
3.5. H3: Task queue méretének hatása	6
3.5.1. Hipotézis és felállítása	6
3.5.2. Eredmények	6
3.5.3. Diskusszió	6
3.6. H4: Task típusok skálázódási profiljának összehasonlítása	6
3.6.1. Hipotézis és felállítása	6
3.6.2. Eredmények	6
3.6.3. Diskusszió	7
3.7. Összegző megfigyelések	7
4. Összefoglalás	8
4.1. A hipotézisek áttekintése	8

4.2. Továbbfejlesztési lehetőségek	8
Irodalomjegyzék	9

1. Feladat ismertetése

A dolgozat célja egy C++ nyelven implementált, több szálon működő lokális cache szerver tervezése és vizsgálata, különös tekintettel a teljesítményoptimalizálási lehetőségekre és azok mérési módszertanára. A rendszer egy párhuzamos feldolgozást alkalmazó szerver architektúrát fog megvalósítani, amely klienskéréseket fogad, azokat worker szálak között osztja szét, és a válasz előállítása során számításigényes feladatokat hajt végre. A számításigényes műveletek célja a CPU-terhelés kontrollált növelése, hogy a különböző konkurens megoldások és cache-stratégiák hatása mérhetővé váljon.

2. Bevezetés

2.1. A téma motivációja

2.2. Elméleti háttér

2.3. A vizsgált hipotézisek

- **H1:** Telítési pont létezése a kliensszám függvényében
- **H2:** Worker pool méretének hatása a teljesítményre
- **H3:** Task queue méretének hatása a latencyre és a reject rátára
- **H4:** Skálázódási profil task típusonként

3. A konkrét munka ismertetése

3.1. Mérési környezet és módszertan

A méréseim során egy HP ZBook 17 G4 típusú laptopot használtam Intel Core i7-7820HQ processzorral, ami négy fizikai processzormaggal rendelkezik, valamint az Intel ún. Simultaneous Multi-Threading (SMT) technológiájával nyolc szálon tesz lehetővé párhuzamos műveletvégzést.

Az operációs rendszer kiválasztásánál a reprodukálhatóság érdekében egy olyan linux-disztribúcióra, az Arch Linuxra esett a választásom, amely arra törekszik, hogy minél inkább kövesse az azt felépítő komponensek eredeti működését, konfigurációját, kevés disztribúcióspecifikus módosítást tartalmazzon.

A méréseimet a Linux kernel 7.0.3 verzióját használva futtattam, a fájlrendszer btrfs volt.

A HTTP kérések generálásához a wrk program 4.2.0-3-es verzióját telepítettem az Arch Linux csomagtárolóiból.

Az általam írt szerver és a wrk program terhelését a processzormagok között a taskset nevű programmal osztottam el, amit a util-linux nevű csomag 2.42-1-es verziójából szereztem be. A nyolc logikai mag közül az első négyet, 0-3-ig a szerver számára osztottam ki, az utolsó kettőt, 6-7 azonosítóval pedig a wrk mérőeszköz számára. Két processzormagot, 4-5 azonosítóval nem érintett a mérésem. Ezt a felosztást annak érdekében valósítottam meg, hogy a szerver mellett futó mérőeszköz ne okozhasson zajt a saját működésével a méréseimben azáltal, hogy kontextusváltást igényelne azokon a processzormagokon, amelyeken a vizsgált program dolgozik. Habár igyekeztem minimalizálni a háttérben futó többi program mennyiségét, az operációs rendszer érdemben való használhatóságához elkerülhetetlen, hogy néhány ilyen jellegű szoftver jelen legyen. Ezek számára azonban az operációs rendszer ki tudta osztani a mérés által nem befolyásolt maradék 4-es és 5-ös processzormagot. A mérés közben, hogy nyomon tudjam követni annak állapotát, a sway nevű ablakkezelő program futott, amelynek rendszerigénye alacsonyabb, mint például egy GNOME vagy KDE környezeté.

A szervert a GNU Compiler Collection (GCC) segítségével fordítottam, a 16.1.1-es verziójával.

A mérési adatokat Python programok segítségével állítottam elő és rendszereztem, valamint dolgoztam fel.

3.2. A szerver implementációja

3.3. H1: Telítési pont a kliensszám függvényében

3.3.1. Hipotézis és felállítása

3.3.2. Eredmények

1. ábra. H1: Throughput és p99 latency együttes viselkedése a kliensszám függvényében.

2. ábra. H1: Latency lineáris karakterisztikája — Little's Law szerinti illeszkedés.

3.3.3. Diskusszió

3.4. H2: Worker pool méretének optimuma

3.4.1. Hipotézis és felállítása

3.4.2. Eredmények

3. ábra. H2: Throughput a worker pool méretének függvényében task típusonként.

4. ábra. H2: Optimum keresése task $\times$ kliensszám szerint, throughput és latency együtt.

3.4.3. Kontrollmérés 2 magos pinninggel

3.4.4. Diskusszió

3.5. H3: Task queue méretének hatása

3.5.1. Hipotézis és felállítása

3.5.2. Eredmények

5. ábra. H3: Latency és reject ráta kompromisszum a queue méret függvényében.

6. ábra. H3: Reject ráta lépcsőfüggvény jellege a queue méret függvényében.

3.5.3. Diskusszió

3.6. H4: Task típusok skálázódási profiljának összehasonlítása

3.6.1. Hipotézis és felállítása

3.6.2. Eredmények

7. ábra. H4: Throughput skálázódás task típusonként.

8. ábra. H4: Normalizált latency skálázódás (single-client érték = 1.0).

3.6.3. Diskusszió

3.7. Összegző megfigyelések

4. Összefoglalás

4.1. A hipotézisek áttekintése

4.2. Továbbfejlesztési lehetőségek

Irodalomjegyzék

Hivatkozások

- [1] J. D. C. Little. A proof for the queuing formula: $L = \lambda W$. *Operations Research*, 9(3):383–387, 1961.
- [2] L. Kleinrock. *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, 1975.
- [3] Boost.Asio dokumentáció. https://www.boost.org/doc/libs/release/doc/html/boost_asio.html. (Letöltve: 2026).
- [4] libcurl — the multiprotocol file transfer library. <https://curl.se/libcurl/>. (Letöltve: 2026).
- [5] W. Glozer. wrk — a HTTP benchmarking tool. <https://github.com/wg/wrk>. (Letöltve: 2026).
- [6] B. Gregg. *Systems Performance: Enterprise and the Cloud*. Pearson, 2. kiadás, 2020.